

CST SDK 接入文档

一、概述

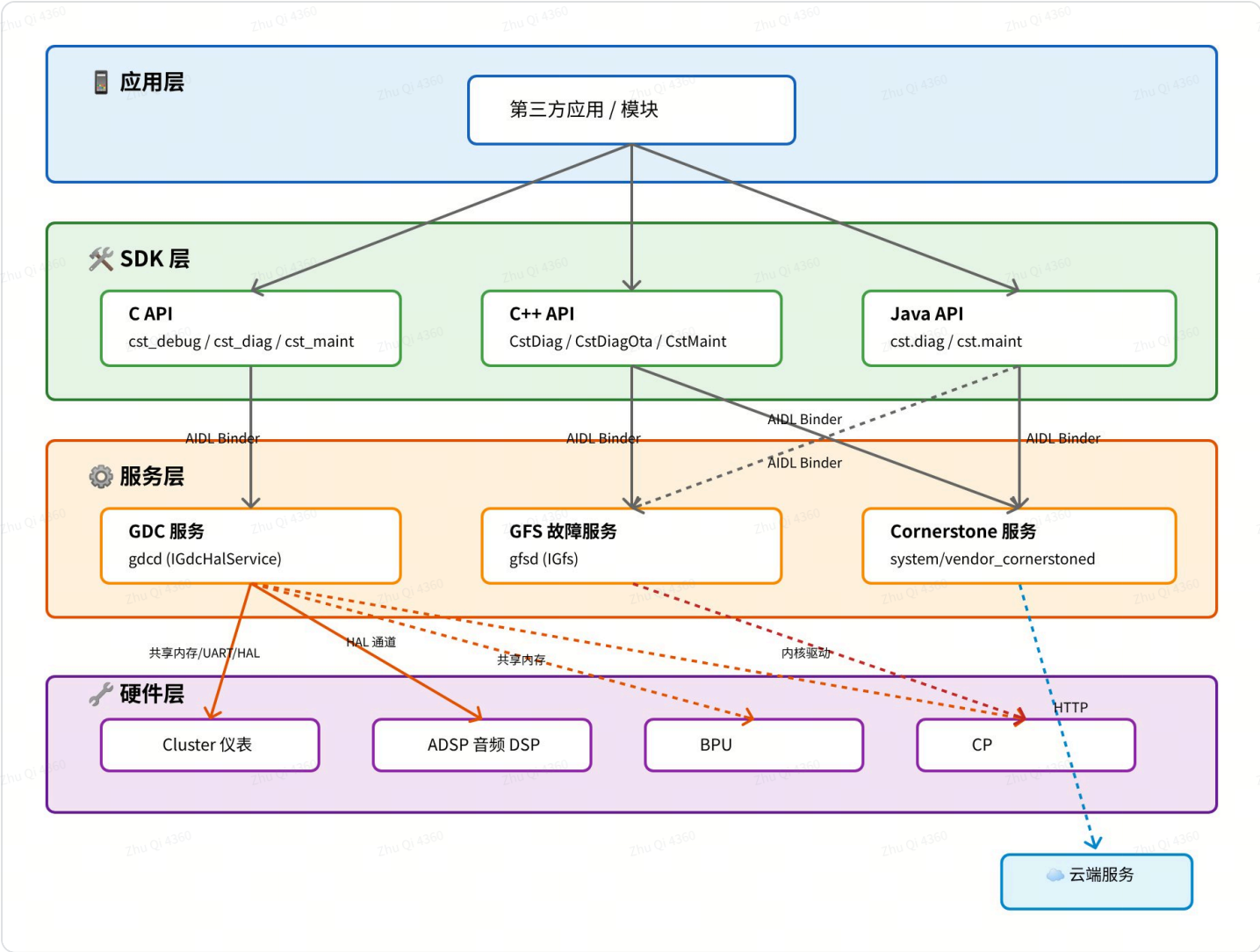
1.1 什么是 CST SDK

CST SDK（Cornerstone SDK）是 Gua 车载 AAOS（Android Automotive OS）平台提供的统一客户端开发套件，封装了与底层诊断通信服务（GDC）、故障服务（GFS）和维护服务（Cornerstone）的交互逻辑，为上层应用提供简洁的 C/C++ 和 Java API。

1.2 三大核心模块

模块	头文件 / 类	核心能力	底层服务
Debug	cst_debug.h	日志上报、命令注册与处理、RawData 传输	GDC (IGdcHalService)
Diag	cst_diag.h / cst_diag_ota.h	故障上报/恢复、故障订阅、故障查询、OTA 升级	GFS (IGfs / IDiagOtaManager)
Maint	cst_maint.h	事件上报、点位上报	Cornerstone (ICornerStoneService)

1.3 系统架构



二、快速接入

2.1 构建依赖配置

2.1.1 C/C++ 项目 (Android.bp)

在模块的 `Android.bp` 中添加以下依赖：

Android.bp 依赖配置

```
1  cc_library_shared {
2      name: "your_module",
3      srcs: ["src/*.cpp"],
4      shared_libs: [
5          "libcst",           // CST SDK 核心库
6          "libbase",
7          "libbinder_ndk",
8          "liblog",
9      ],
```

```

10 }
11
12 // 如果使用 Debug 模块（非 PAD 设备），额外依赖：
13 // shared_libs: [
14 //     "vendor.gua.hardware.gdc-V1-ndk",    // GDC AIDL 库
15 // ]
16
17 // 如果使用 Diag 模块，额外依赖：
18 // shared_libs: [
19 //     "gua.gfs-V1-ndk",                    // GFS AIDL 库
20 // ]
21
22 // 如果使用 Maint 模块，额外依赖：
23 // shared_libs: [
24 //     "vendor.gua.cornerstone.transfer-V1-ndk", // Cornerstone AIDL 库
25 // ]

```

！ 注意：Debug 模块在 PAD 设备上不可用。 SDK 内部通过 Soong 插件（`cst.go`）在编译时根据 `deviceProduct` 自动判断：PAD 设备（`gua1_pad`）不包含 Debug 源码和 GDC 依赖，仅添加 `-DGUA_PAD=1` 编译标志。

2.1.2 Java 项目 (Android.bp)

Java 依赖配置

```

1  java_library {
2      name: "your_java_module",
3      srcs: ["src/**/*.java"],
4      libs: [
5          "cst-sdk",          // CST SDK Java 库
6      ],
7      static_libs: [
8          "gua.gfs-V1-java",  // GFS Java AIDL 库 (Diag 模块)
9      ],
10 }

```

2.1.3 Linux 平台 (CMakeLists.txt)

2.1.4 APK 项目 (Gradle)

对于使用 Gradle 构建的 Android APK 应用，接入 CST SDK 需要以下步骤：

！ 前提条件： CST SDK 的 Java 层使用了 Android 平台内部 API（`android.os.ServiceManager`），APK 必须具有 `system` 或 `privileged` 签名才能正常调用。普通第三方应用无法使用 Java SDK。

一、获取 SDK 依赖

CST SDK 当前以 AOSP 源码形式提供，不发布到 Maven 仓库。需要手动将编译产物引入 Gradle 项目：

方式 A：源码编译（推荐）

在 AOSP 全量编译环境中，CST SDK 的相关模块会编译生成以下 JAR 文件：

编译产物	AOSP 模块名	用途
<code>cst-sdk.jar</code>	<code>cst-sdk</code>	CST SDK Java 核心库
<code>gua.gfs-V1-java.jar</code>	<code>gua.gfs-V1-java</code>	GFS AIDL 接口 (Diag 模块依赖)
<code>vendor.gua.cornerstone.transfer-V1-java.jar</code>	<code>vendor.gua.cornerstone.transfer-V1-java</code>	Cornerstone AIDL 接口 (Maint 模块依赖)

编译命令：

```
AOSP 编译 SDK 产物
1  # 编译 CST SDK 及其依赖
2  source build/envsetup.sh
3  lunch <目标产品>
4
5  # 编译所有相关模块
6  m cst-sdk gua.gfs-V1-java vendor.gua.cornerstone.transfer-V1-java
7
8  # 产物路径
9  # out/target/product/<device>/system_ext/framework/cst-sdk.jar
10 # out/target/product/<device>/system_ext/framework/gua.gfs-V1-java.jar
11 # out/target/product/<device>/system_ext/framework/vendor.gua.cornerstone.transfer-V1-java.jar
```

方式 B：从设备提取

如果已有烧录了 Gua 系统的设备，可以从设备上提取 JAR 文件：

从设备提取 SDK

```
1  # 从设备拉取 JAR 文件
2  adb pull /system_ext/framework/cst-sdk.jar ./
3  adb pull /system_ext/framework/gua.gfs-V1-java.jar ./
4  adb pull /system_ext/framework/vendor.gua.cornerstone.transfer-V1-java.jar ./
```

二、Gradle 配置

将编译产物作为本地 JAR 依赖引入项目：

1. 放置 JAR 文件

项目目录结构

```
1  app/
2    libs/
3      cst-sdk.jar
4      gua.gfs-V1-java.jar
5      vendor.gua.cornerstone.transfer-V1-java.jar
6  src/main/
7    AndroidManifest.xml
8    java/...
```

2. 配置 build.gradle

app/build.gradle 配置

```
1  android {
2      compileSdkVersion 33 // 与目标 AAOS 版本一致
3
4      defaultConfig {
5          applicationId "com.example.yourapp"
6          minSdkVersion 28
7          targetSdkVersion 33
8      }
9  }
10
11  dependencies {
12      // CST SDK 核心库
13      implementation files("libs/cst-sdk.jar")
14
15      // GFS AIDL 依赖 (Diag 模块必需)
16      implementation files("libs/gua.gfs-V1-java.jar")
17  }
```

```
18 // Cornerstone AIDL 依赖 (Maint 模块必需)
19 implementation files("libs/vendor.gua.cornerstone.transfer-V1-java.jar")
20 }
```

 **隐藏 API 问题：** CST SDK 内部调用了 `android.os.ServiceManager` 等 Android 隐藏 API。在 Gradle 编译时可能会遇到 "cannot find symbol" 错误。解决方案：

✅ 方案1：使用系统 SDK（推荐）

在 AOSP 环境下使用 `Android.mk` 或 `Android.bp` 编译 APK，自动可访问隐藏 API。这是 Gua 平台推荐的方式。

方案2：添加 stub JAR

在 Gradle 中将包含隐藏 API 的 `android.jar`（从 AOSP out 目录获取）作为 `compileOnly` 依赖，替换标准 SDK 的 `android.jar`。

三、AndroidManifest.xml 配置

APK 必须声明为系统应用并配置适当的权限：

AndroidManifest.xml 配置

```
1 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
2     package="com.example.yourapp"
3     android:sharedUserId="android.uid.system">
4
5     <application
6         android:label="Your App"
7         android:systemUserOnly="true">
8
9         <!-- 声明使用 GFS 服务 (Diag 模块) -->
10        <uses-library
11            android:name="gua.gfs-V1-java"
12            android:required="true" />
13
14        <!-- 声明使用 Cornerstone 传输层 (Maint 模块) -->
15        <uses-library
16            android:name="vendor.gua.cornerstone.transfer-V1-java"
17            android:required="true" />
18
19    </application>
20 </manifest>
```

四、签名配置

 **GUA 签名验证：**CST SDK 的 Java `CstDiag` 类内置了 GUA 证书签名验证（`isGUASignature()`）。APK 必须使用 GUA 平台签名证书签名才能通过验证。使用非 GUA 签名的 APK 调用 `CstDiag` 会被拒绝。

CstMaint 无签名限制：维护模块（`CstMaint`）不进行签名验证，任何系统应用均可使用。

在 Gradle 中配置签名：

签名配置

```
1  android {
2      signingConfigs {
3          platform {
4              storeFile file("platform.jks")
5              storePassword "your_store_password"
6              keyAlias "platform"
7              keyPassword "your_key_password"
8          }
9      }
10
11     buildTypes {
12         release {
13             signingConfig signingConfigs.platform
14         }
15         debug {
16             signingConfig signingConfigs.platform
17         }
18     }
19 }
```

五、预装到系统镜像

由于 APK 需要系统签名和 `system_ext` 分区的 JAR 依赖，必须预装到系统镜像中：

Android.bp 预装定义

```
1  android_app_import {
2      name: "YourApp",
3      apk: "YourApp.apk",
4      system_ext_specific: true,
5      privileged: true,          // 放入 priv-app 目录
6      certificate: "platform",   // 使用平台签名重签名
7      dex_preopt: {
8          enabled: false,        // 可选：禁用 dex 优化以加速开发
9      },
```

```
10     required: [
11         "cst-sdk",           // 确保依赖的 JAR 已安装
12         "gua.gfs-V1-java",
13         "vendor.gua.cornerstone.transfer-V1-java",
14     ],
15 }
```

六、完整 Gradle 接入示例

完整的 app/build.gradle 示例

```
1  plugins {
2      id "com.android.application"
3  }
4
5  android {
6      namespace "com.example.guasdkdemo"
7      compileSdk 33
8
9      defaultConfig {
10         applicationId "com.example.guasdkdemo"
11         minSdk 28
12         targetSdk 33
13         versionCode 1
14         versionName "1.0"
15     }
16
17     signingConfigs {
18         platform {
19             storeFile file("../certs/platform.jks")
20             storePassword System.getenv("KEYSTORE_PASSWORD")
21             keyAlias "platform"
22             keyPassword System.getenv("KEY_PASSWORD")
23         }
24     }
25
26     buildTypes {
27         release {
28             signingConfig signingConfigs.platform
29             minifyEnabled false
30         }
31         debug {
32             signingConfig signingConfigs.platform
33         }
34     }
35 }
```



```

36
37 dependencies {
38     implementation files("libs/cst-sdk.jar")
39     implementation files("libs/gua.gfs-V1-java.jar")
40     implementation files("libs/vendor.gua.cornerstone.transfer-V1-java.jar")
41     implementation "androidx.appcompat:appcompat:1.6.1"
42 }

```

APK 中使用 SDK 的示例代码

```

1  import cst.diag.CstDiag;
2  import cst.maint.CstMaint;
3
4  public class MainActivity extends Activity {
5      @Override
6      protected void onCreate(Bundle savedInstanceState) {
7          super.onCreate(savedInstanceState);
8
9          // Diag 模块: 上报故障 (需要 GUA 签名)
10         CstDiag.submitFault(100010001, CstDiag.SEVERITY_ERROR,
11             "sensor_timeout", "{\"sensor_id\": 3}");
12
13         // Maint 模块: 上报维护事件 (无签名限制)
14         CstMaint.submitEvent(100010001, "{\"action\": \"system_boot\"}");
15
16         // Maint 模块: 上报维护标记 (无签名限制)
17         CstMaint.submitMark(100020001, "{\"mark\": \"init_complete\"}");
18     }
19 }

```

SDK 也支持在 Linux 平台上构建（用于 Cluster 侧开发）：

CMake 依赖配置

```

1  find_package(cst REQUIRED)
2  target_link_libraries(your_target cst)

```

2.2 头文件引入

模块	C/C++ 头文件	Java 包
Debug	<pre>#include <cst_debug.h></pre>	—

Diag（故障）	#include <cst_diag.h>	cst.diag.CstDiag
Diag（OTA）	#include <cst_diag_ota.h>	—
Maint	#include <cst_maint.h>	cst.maint.CstMaint

三、Debug 模块

 **Debug 模块**提供异构子系统（Cluster、ADSP、BPU、CP 等）的日志上报、远程命令注册与处理、RawData 共享内存传输能力。仅 IVI 和 Cluster 平台可用，PAD 设备不可用。

3.1 子系统 ID 定义

每个异构芯片子系统分配唯一 ID，用于标识日志和命令的来源/目标：

ID	常量名	说明
1	CST_SUBSYS_ID_CLUSTER	仪表盘（Cluster）
2	CST_SUBSYS_ID_ADSP1	音频 DSP 1
3	CST_SUBSYS_ID_ADSP2	音频 DSP 2
4	CST_SUBSYS_ID_ADSP3	音频 DSP 3
5	CST_SUBSYS_ID_BPU1	BPU 神经网络处理器 1
6	CST_SUBSYS_ID_BPU2	BPU 神经网络处理器 2
7	CST_SUBSYS_ID_BPU3	BPU 神经网络处理器 3
8	CST_SUBSYS_ID_BPU4	BPU 神经网络处理器 4
9	CST_SUBSYS_ID_CP0	功能安全 CP 核 0
10	CST_SUBSYS_ID_CP1_C0	功能安全 CP1 核 C0
11	CST_SUBSYS_ID_CP1_C1	功能安全 CP1 核 C1

3.2 日志上报 API

3.2.1 文本日志上报

文本日志上报

```
1 // 基础日志上报
2 int cst_debug_submit_log(int subsys_id, const char* log);
3
4 // 带标签的日志上报 (支持按 tag 分组缓存和分发)
5 int cst_debug_submit_log_with_tag(int subsys_id, const char* tag, const char* log);
```

参数说明:

- subsys_id：子系统 ID，参考 3.1 节定义
- tag：日志标签，用于分组（如 "tombstone"、"kernel"）
- log：日志文本内容

3.2.2 二进制日志上报

二进制日志上报

```
1 // 上报二进制日志缓冲区
2 int cst_debug_submit_log_buffer(int subsys_id, const uint8_t* buf, int size);
3
4 // 带标签上报二进制日志
5 int cst_debug_submit_log_buffer_with_tag(int subsys_id, const char* tag,
6                                           const uint8_t* buf, int size);
```

3.2.3 返回码

返回值	常量	说明
0	CST_DEBUG_RET_OK	成功
1	ERR_INVALID_PARAM	参数无效
2	ERR_SUBSYSID	子系统 ID 无效
3	ERR_DATA_LENGTH	数据长度超限
4	ERR_CHN_CONFIG	通道配置错误
5	ERR_RAWDATA_SHM	RawData 共享内存错误

6	ERR_OPEN_FILE_FAILED	文件打开失败
7	ERR_SERVICE_OFFLINE	服务离线

3.3 命令注册与处理 API

Debug 模块支持向 GDC 注册命令处理器，接收来自远程（如 `gremoteshell` 工具）的命令并返回结果。

3.3.1 命令处理器回调类型

命令处理器回调定义

```
1 typedef int (*cst_debug_cmd_handle_func)(
2     const char* cmd_name,      // 命令名称
3     const char* cmd_para,      // 命令参数
4     char* result,              // 输出结果缓冲区
5     int result_size            // 缓冲区大小
6 );
7 // 返回 0 表示命令执行成功，非 0 表示失败
```

3.3.2 注册命令处理器

注册命令处理器

```
1 // 注册单个命令处理器
2 int cst_debug_register_cmd_handler(
3     int subsys_id,
4     const char* cmd_name,      // 命令名，如 "hello"
5     cst_debug_cmd_handle_func handler, // 回调函数
6     bool is_sync              // true=同步处理，false=异步处理
7 );
8
9 // 批量注册命令处理器
10 int cst_debug_register_cmd_handlers(
11     int subsys_id,
12     const char** cmd_names,    // 命令名数组
13     int num,                  // 命令数量
14     cst_debug_cmd_handle_func handler, // 统一回调函数
15     bool is_sync
16 );
```

 **同步 vs 异步模式：**同步模式下，命令在 GDC 服务线程中直接执行回调，回调返回后结果立即返回给调用方。异步模式下，回调仅触发通知，需在处理完成后调用 `cst_debug_submit_response()` 主动上报结果。

3.3.3 异步命令响应上报

异步命令响应

```
1 // 异步模式下，处理完成后上报结果
2 int cst_debug_submit_response(
3     int subsys_id,
4     const char* cmd_str, // 对应的命令名
5     int ret, // 执行结果码 (0=成功)
6     const char* result // 结果文本
7 );
```

3.4 RawData 传输 API

RawData 机制允许子系统通过共享内存高效传输大块二进制数据（如 coredump、内存快照等）。

3.4.1 共享内存方式

RawData 共享内存传输

```
1 // 1. 获取共享内存缓冲区
2 void* addr = NULL;
3 int size = 0;
4 int ret = cst_debug_get_rawdata_buffer(subsys_id, &addr, &size);
5
6 // 2. 将数据写入共享内存
7 if (addr && size > 0) {
8     memcpy(addr, your_data, your_data_size);
9 }
10
11 // 3. 提交 RawData
12 cst_debug_submit_rawdata(subsys_id, rawdata_id, offset, your_data_size);
13
14 // 带标签提交
15 cst_debug_submit_rawdata_with_tag(subsys_id, tag, offset, your_data_size);
```

3.4.2 文件方式

RawData 文件传输

```
1 // 直接提交文件作为 RawData
2 cst_debug_submit_rawdata_file(subsys_id, rawdata_id, "/path/to/file");
3
4 // 带标签提交文件
5 cst_debug_submit_rawdata_file_with_tag(subsys_id, tag, "/path/to/file");
```

3.5 Debug 模块完整示例

Debug 模块 C 语言示例

```
1 #include <cst_debug.h>
2 #include <pthread.h>
3 #include <string.h>
4
5 // 命令处理回调
6 static int on_cmd(const char* cmd_name, const char* cmd_para,
7                  char* result, int result_size) {
8     if (strcmp(cmd_name, "hello") == 0) {
9         snprintf(result, result_size, "Hello from subsys!");
10        return 0;
11    } else if (strcmp(cmd_name, "wait") == 0) {
12        // 异步命令: 先返回, 稍后通过 submit_response 上报
13        return 1; // 返回非 0 表示异步处理中
14    }
15    return -1;
16 }
17
18 void init_debug() {
19     int subsys = CST_SUBSYS_ID_CLUSTER;
20
21     // 注册命令处理器
22     const char* cmds[] = {"hello", "hi", "wait", "other"};
23     cst_debug_register_cmd_handlers(subsys, cmds, 4, on_cmd, true);
24
25     // 上报文本日志
26     cst_debug_submit_log(subsys, "Application started");
27
28     // 上报带标签日志
29     cst_debug_submit_log_with_tag(subsys, "startup", "Init complete");
30
31     // 上报 RawData (共享内存方式)
32     void* addr = NULL;
33     int size = 0;
34     if (cst_debug_get_rawdata_buffer(subsys, &addr, &size) == 0) {
35         memcpy(addr, "raw binary data", 15);
36         cst_debug_submit_rawdata(subsys, 1, 0, 15);
```

```
37     }
38
39     // 上报 RawData (文件方式)
40     cst_debug_submit_rawdata_file(subsys, 1, "/data/local/tmp/coredump.bin");
41 }
```

四、Diag 诊断模块

 **Diag 模块**提供故障（Fault）的全生命周期管理，包括上报、恢复、订阅、查询，以及 OTA 升级回调机制。底层对接 GFS（Global Fault Service）。

4.1 故障严重等级

值	常量	说明
0	CST_DIAG_SEVERITY_WARN	警告级别
1	CST_DIAG_SEVERITY_ERROR	错误级别
2	CST_DIAG_SEVERITY_FATAL	致命级别（会同步至内核驱动和云端）

4.2 C++ API（推荐）

C++ API 通过静态方法类 `cst::diag::CstDiag` 提供完整的故障管理能力：

4.2.1 故障上报与恢复

故障上报与恢复

```
1  #include <cst_diag.h>
2
3  using namespace cst::diag;
4
5  // 创建故障对象并上报
6  auto fault = CstDiag::createFault(
7      100010001,                // fault_id: 故障 ID
8      CST_DIAG_SEVERITY_ERROR,  // severity: 严重等级
9      "sensor_timeout",         // reason: 故障原因
```

```
10     "{ \"sensor_id\": 3}" // payload: 附加信息 (JSON 格式)
11 );
12 CstDiag::submitFault(fault);
13
14 // 上报故障恢复
15 CstDiag::submitRecovery(100010001);
```

4.2.2 故障订阅

故障订阅

```
1  #include <cst_diag.h>
2  #include <functional>
3
4  using namespace cst::diag;
5
6  // 订阅单个故障
7  CstDiag::subscribeFault(100010001, [](const DiagFault& fault) {
8      printf("Fault %d occurred: %s\n", fault.id, fault.reason.c_str());
9  });
10
11 // 批量订阅多个故障
12 std::vector<int> faultIds = {100010001, 100010002, 100010003};
13 CstDiag::subscribeFaults(faultIds, [](const DiagFault& fault) {
14     printf("Fault %d: %s\n", fault.id, fault.reason.c_str());
15 });
16
17 // 取消订阅
18 CstDiag::unsubscribeFaults(faultIds);
```

4.2.3 故障查询

故障查询

```
1  // 查询所有故障
2  auto allFaults = CstDiag::queryAllFaults();
3
4  // 查询单个故障详情
5  auto faultInfo = CstDiag::queryFaultInfo(100010001);
6
7  // 查询故障计数 (返回 [总数, CPC计数])
8  auto counts = CstDiag::queryFaultCount(100010001);
9
10 // 按时间范围查询
11 auto faults = CstDiag::queryFaultsByTime(beginTimestamp, endTimestamp);
```



```
12
13 // 检查故障是否正在发生
14 bool occurred = CstDiag::isFaultOccurred(100010001);
```

4.3 C API

对于不支持 C++ 的场景，提供纯 C 接口：

Diag C API

```
1 #include <cst_diag.h>
2
3 // 上报故障
4 cst_diag_submit_fault(
5     100010001, // fault_id
6     CST_DIAG_SEVERITY_ERROR, // severity
7     "sensor_timeout", // reason
8     "{ \"sensor_id\": 3}" // payload
9 );
10
11 // 上报故障恢复
12 cst_diag_submit_recovery(100010001);
```

! C API 限制： C API 仅提供故障上报和恢复接口，订阅和查询功能需使用 C++ API 或 Java API。

4.4 Java API

Java Diag API

```
1 import cst.diag.CstDiag;
2
3 // 上报故障
4 CstDiag.submitFault(100010001, CstDiag.SEVERITY_ERROR,
5     "sensor_timeout", "{ \"sensor_id\": 3}");
6
7 // 上报故障恢复
8 CstDiag.submitRecovery(100010001);
9
10 // 订阅故障
11 CstDiag.subscribe(100010001, fault -> {
12     Log.i("Diag", "Fault " + fault.id + ": " + fault.reason);
13 });
14
```

```

15 // 查询所有故障
16 List<DiagFault> allFaults = CstDiag.queryFaultsAll();
17
18 // 查询故障是否正在发生
19 boolean occurred = CstDiag.isFaultOccurred(100010001);

```

! Java SDK 签名验证：Java 层 `CstDiag` 内置 GUA 签名验证（`isGUASignature()`），仅允许 GUA 签名的 APK 使用。非 GUA 签名的应用调用会被拒绝。

4.5 Diag OTA 升级模块

Diag OTA 模块用于处理通过 UDS 诊断协议触发的 ECU 固件升级流程。OTA 进程需要实现 `CstDiagOtaCallback` 抽象接口并注册到 SDK。

4.5.1 OTA 事件类型

事件	常量	说明
1	<code>EVENT_CHECK_DOWNLOAD_CONDITION</code>	检查下载条件
2	<code>EVENT_CHECK_PROGRAMMING_DEPENDENCY</code>	检查刷写依赖
3	<code>EVENT_VERIFY_SECURITY_SIGNATURE</code>	验证安全签名
4	<code>EVENT_PREVENT_PARTITION_SWITCHOVER</code>	阻止分区切换
5	<code>EVENT_START_VERSION_INSTALL</code>	开始版本安装
6	<code>EVENT_SWITCH_VERSION</code>	切换版本
7	<code>EVENT_QUERY_SWITCHOVER_RESULT</code>	查询切换结果
8	<code>EVENT_GET_INSTALL_PROGRESS</code>	获取安装进度
9	<code>EVENT_VERIFY_POST_INSTALL_RESULT</code>	验证安装后结果

4.5.2 OTA 回调接口

OTA 回调接口实现

```

1 #include <cst_diag_ota.h>
2
3 class MyOtaCallback : public cst::diag::CstDiagOtaCallback {

```

```

4 public:
5     // 统一事件分发（必须实现）
6     DiagOtaResponse onEvent(const std::string& taskId, int event,
7                             const std::vector<uint8_t>& param) override {
8         DiagOtaResponse resp;
9         resp.taskId = taskId;
10        resp.event = event;
11        switch (event) {
12            case EVENT_CHECK_DOWNLOAD_CONDITION:
13                resp.resultCode = RESULT_OK;
14                break;
15            case EVENT_START_VERSION_INSTALL:
16                resp.resultCode = RESULT_OK;
17                break;
18            default:
19                resp.resultCode = RESULT_NOT_SUPPORTED;
20                break;
21        }
22        return resp;
23    }
24
25    void onPackageTransferStart(const std::string& taskId,
26                               const TransferMetaInfo& meta) override {}
27    void onPackageChannel(const std::string& taskId, int channelFd) override {}
28    void onPackageBlockReady(const std::string& taskId, int blockSize)
29    override {}
30    void onPackageTransferComplete(const std::string& taskId, int result)
31    override {}
32    void onError(const std::string& taskId, int errorCode,
33                const std::string& message) override {}
34 };

```

4.5.3 OTA 注册与结果上报

OTA 注册与结果上报

```

1 // 注册 OTA 回调
2 MyOtaCallback callback;
3 cst::diag::CstDiagOta::registerCallback(&callback);
4
5 // 确认数据块接收
6 cst::diag::CstDiagOta::confirmBlockReceived(taskId, true);
7
8 // 释放共享内存通道
9 cst::diag::CstDiagOta::releaseChannel(taskId, true);
10

```

```
11 // 异步上报 OTA 操作结果
12 DiagOtaResponse response;
13 response.taskId = taskId;
14 response.event = EVENT_START_VERSION_INSTALL;
15 response.resultCode = RESULT_OK;
16 response.progress = 50;
17 cst::diag::CstDiagOta::reportResult(response);
18
19 // 注销回调
20 cst::diag::CstDiagOta::unregisterCallback();
```

4.5.4 OTA 结果码

值	常量	说明
-1	RESULT_PENDING	处理中（异步）
0	RESULT_OK	操作成功
1	RESULT_FAILED	操作失败
2	RESULT_TIMEOUT	操作超时
3	RESULT_BUSY	当前繁忙
4	RESULT_INVALID_PARAM	参数无效
5	RESULT_NOT_SUPPORTED	不支持的操作
6	RESULT_CONDITION_NOT_MET	条件不满足
7	RESULT_SECURITY_DENIED	安全校验失败

五、Maint 维测模块

 **Maint 模块**提供维护事件（Event）和维护标记（Mark）的上报能力。事件用于记录系统中的重要状态变化，标记用于标记关键时间节点或触发条件。底层对接 Cornerstone 维护服务。

5.1 Event ID 约束

! Event ID 必须是 9 位数字，范围 100000000 ~ 999999999。不在该范围的 ID 会被 SDK 拒绝。

5.2 C++ API

Maint C++ API

```
1  #include <cst_maint.h>
2
3  using namespace cst::maint;
4
5  // 获取单例
6  auto& maint = CstMaint::getInstance();
7
8  // 上报维护事件 (Cluster 版本)
9  maint.submitEvent(100010001, "{\"action\": \"system_boot\"}");
10
11 // 上报维护事件 (带文件列表, 非 Cluster 版本)
12 std::vector<std::string> files = {"/data/local/tmp/log1.txt",
13                                   "/data/local/tmp/log2.txt"};
14 maint.submitEvent(100010002, "{\"action\": \"crash_dump\"}", files);
15
16 // 上报维护标记
17 maint.submitMark(100020001, "{\"mark\": \"shutdown_start\"}");
```

5.3 C API

Maint C API

```
1  #include <cst_maint.h>
2
3  // 上报维护事件
4  cst_maint_submit_event(100010001, "{\"action\": \"system_boot\"}");
5
6  // 上报维护标记
7  cst_maint_submit_mark(100020001, "{\"mark\": \"shutdown_start\"}");
```

5.4 Java API

Maint Java API

```
1  import cst.maint.CstMaint;
2
```

```
3 // 上报维护事件
4 CstMaint.submitEvent(100010001, "{\"action\": \"system_boot\"}");
5
6 // 上报维护事件（带文件列表）
7 List<String> files = Arrays.asList("/data/local/tmp/log1.txt");
8 CstMaint.submitEvent(100010002, "{\"action\": \"crash_dump\"}", files);
9
10 // 上报维护标记
11 CstMaint.submitMark(100020001, "{\"mark\": \"shutdown_start\"}");
```

5.5 限流机制

 **Mark 限流规则：**Maint 模块对 Mark 上报实施令牌桶限流，每个 Event ID 独立限流。限流参数：**1 次/秒**，最大桶容量 100。超频 Mark 会被丢弃，并自动上报一个 eventId= 100010006 的警告标记。

六、底层服务说明

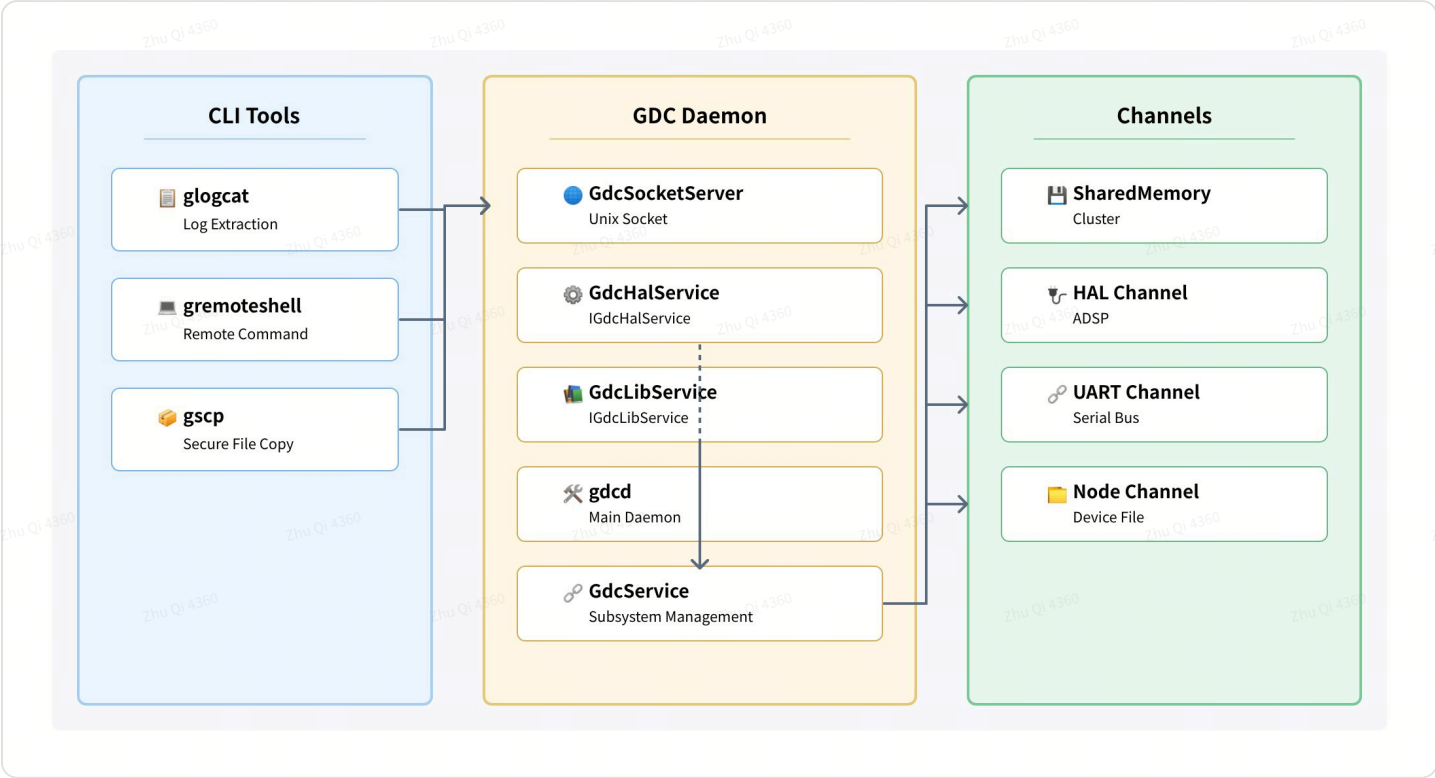
6.1 服务总览

服务名	进程	注册的 AIDL 接口	启动时机
GDC	gdcd	vendor.gua.hardware.gdc.IGdcHalService/default vendor.gua.hardware.gdc.IGdcLibService/default vendor.gua.hardware.gdc.IFakeShmService/default	post-fs
GFS	gfsd	vendor.gua.hardware.gfs.IGfs/default vendor.gua.hardware.gfs.IGdcProxy/default vendor.gua.hardware.gfs.IDiagOtaManager/default	post-fs-data
Cornerstone	system_cornerstoned	vendor.gua.cornerstone.ICornerStoneService/system	post-fs-data
Cornerstone			class hal

vendor_corne rstoned	vendor.gua.cornerstone.IVendorCorn erStoneService/vendor
-------------------------	---

6.2 GDC 服务（Global Diagnostic Communication）

6.2.1 服务架构



6.2.2 GDC 通道类型

通道类型	类名	说明
SHM	ShmChannel	与 Cluster 通过物理共享内存通信
FakeShm	FakeShmChannel	伪共享内存（Binder 模拟，用于测试）
HAL	HalChannel	HAL 层通信（ADSP 子系统）
UART	UartChannel	UART 串口通信
GIPC	GIPCChannel	GIPC 进程间通信
Node	NodeChannel	设备文件读取（如 /dev/log_bl31_cluster）
PStore	PStoreChannel	Panic Store 读取（/sys/fs/pstore/）

6.2.3 CLI 工具

glogcat — 日志提取

glogcat 用法

```
1 # 查看 Cluster 日志
2 glogcat -b cluster
3
4 # 查看 ADSP 日志
5 glogcat -b adsp1
6
7 # 按 tag 过滤
8 glogcat -b cluster --
  tag=tombstone
9
10 # 转储 rawdata
11 glogcat -b cluster -t rawdata \
12     --rawdata-dir=/data/local/tmp/
13
14 # 输出到文件
15 glogcat -b all -f /tmp/gdc.log
```

gremoteshell — 远程命令

gremoteshell 用法

```
1 # 单次命令执行
2 gremoteshell -s cluster -c "ls
  /tmp"
3
4 # 交互模式
5 gremoteshell -s adsp1
6
7 # 指定所有子系统
8 gremoteshell -s all -c "uname -a"
```

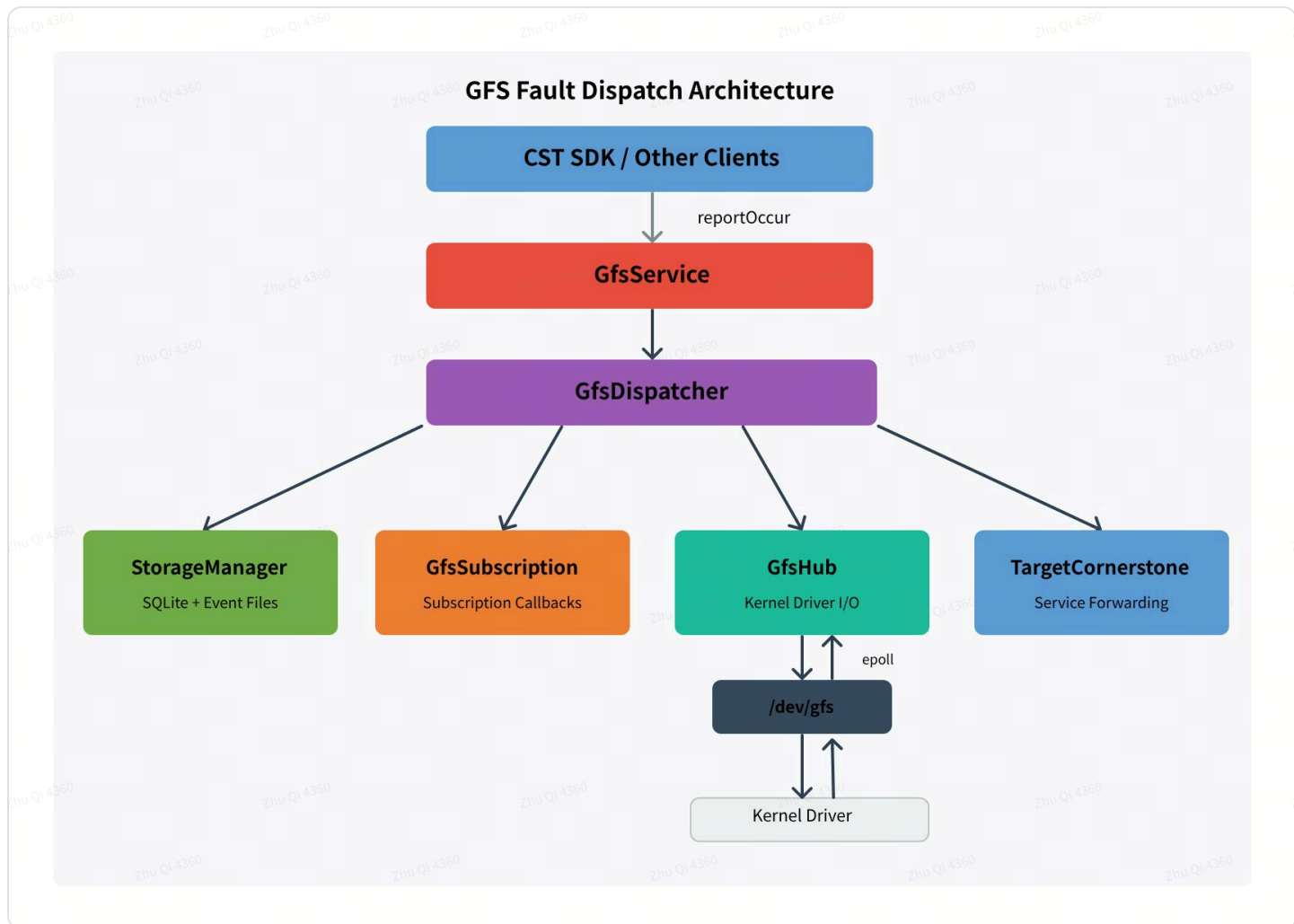
gscp — 安全文件拷贝

gscp 用法

```
1 # IVI -> Cluster
2 gscp /mnt/file.txt cluster:/tmp/
3
4 # Cluster -> IVI
5 gscp cluster:/tmp/file.txt /mnt/
6
7 # 递归拷贝目录
8 gscp -r /mnt/dir/ cluster:/tmp/
```

6.3 GFS 服务 (Global Fault Service)

6.3.1 故障分发架构

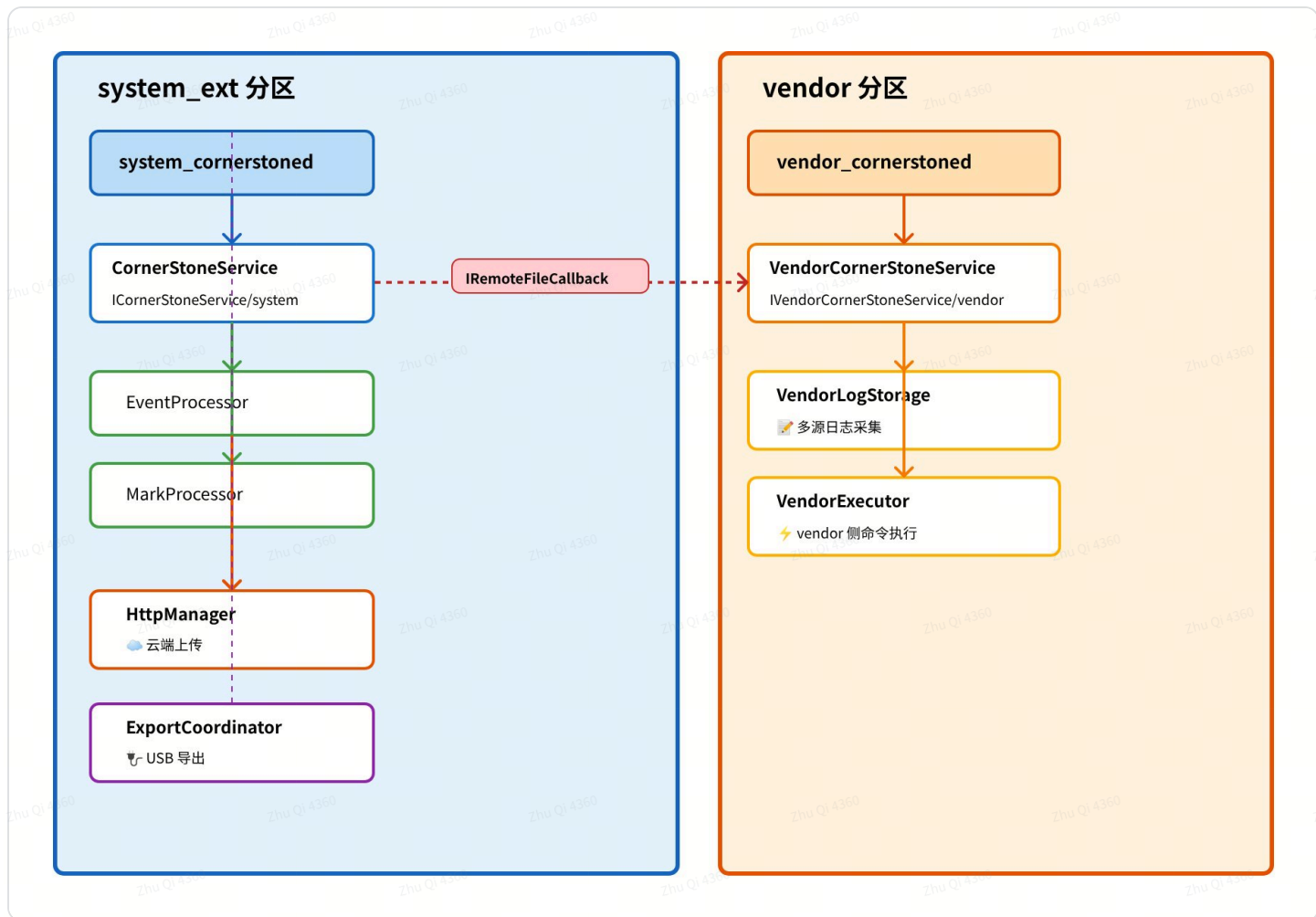


6.3.2 故障数据流

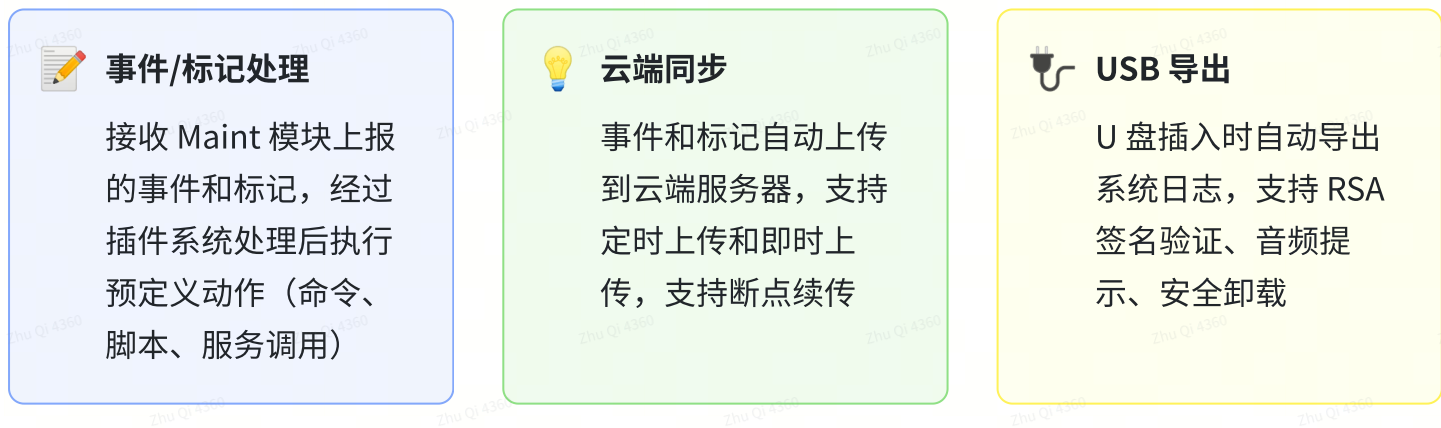
1. 客户端通过 `IGfs.reportOccur()` 上报故障
2. **GfsDispatcher** 将故障分发到 4 个目标方向：
 - **StorageManager**: 持久化到 SQLite 数据库 + 事件文本文件
 - **GfsSubscription**: 通知所有订阅该故障的回调
 - **GfsHub**: FATAL 级别故障写入内核驱动 (`/dev/gfs`)
 - **TargetCornerstone**: 转发到 Cornerstone 维护服务 (事件上报 + 云端同步)
3. 故障恢复时, **GfsDispatcher** 同样通知 4 个方向

6.4 Cornerstone 服务

6.4.1 双进程架构



6.4.2 核心功能



七、SDK 内部机制

7.1 服务自动重连

SDK 的所有三个模块（Debug、Diag、Maint）都内置了服务死亡检测和自动重连机制：

1. SDK 通过 `AServiceManager_registerForServiceNotifications` 注册服务可用性监听
2. Binder 连接上注册 `DeathRecipient`，服务进程崩溃时收到通知

3. 服务恢复后自动重建连接，并 flush 缓存数据

7.2 离线缓存

 **推荐：** SDK 在服务离线期间自动缓存数据，服务恢复后自动 flush，开发者无需关心服务可用性。

模块	缓存策略	容量限制
Debug	按 tag 分组的日志缓存队列	每个 tag 独立线程发送
Diag	故障缓存（服务离线时缓存，恢复后 flush）	—
Maint	Event 队列 + Mark 队列	Event: 500 个，Mark: 1000 个

7.3 限流保护

Maint 模块的 Mark 使用令牌桶算法限流：

- 每个 Event ID 独立的令牌桶
- 速率：**1 token/秒**
- 桶容量：**100 tokens**
- 超频时自动上报 eventId= 100010006 的告警标记

7.4 平台差异

特性	IVI 平台	Cluster 平台	PAD 平台
Debug 模块	✅ 完整支持	✅ 完整支持	❌ 不可用
Diag 模块	✅ AIDL Binder	✅ AIDL Binder	✅ AIDL Binder
Maint 模块	✅ ICornerStoneService	✅ maint_service	✅ ICornerStoneService
Diag OTA	✅ 支持	✅ 支持	✅ 支持
GDC 配置	gdc_ivi.xml	gdc_cluster.xml	—

八、AIDL 接口参考

8.1 GDC AIDL 接口

8.1.1 IGdcHalService

Debug 模块使用的 GDC HAL 服务接口：

IGdcHalService.aidl

```
1  interface IGdcHalService {
2      oneway void submitLog(int subsysId, String tag, String log);
3      oneway void submitCstBuffer(int subsysId, int type, in byte[] buffer);
4      void registerCallBack(int subsysId, String cmdName,
5                             ICmdHandler handler, boolean isSync);
6      oneway void submitResponse(int subsysId, String cmdName,
7                                 int result, String content);
8      void registerRawDataInfo(int subsysId, in ParcelFileDescriptor fd, int
size);
9      oneway void submitRawdata(int subsysId, String tag, int offset, int size);
10     oneway void submitRawdataFile(int subsysId, String tag, String filePath);
11 }
```

8.1.2 IGdcLibService

libgdc 客户端库使用的订阅/命令接口：

IGdcLibService.aidl

```
1  interface IGdcLibService {
2      void subscribeLog(int subsysId, String tag, IGdcLibHandler dispatch);
3      void unsubscribeLog(IGdcLibHandler dispatch);
4      void subscribeRawdata(int subsysId, String tag, IGdcLibHandler dispatch);
5      void unsubscribeRawdata(IGdcLibHandler handle);
6      void sendCmd(int subsysId, String cmd, IGdcLibHandler dispatch);
7  }
```

8.2 GFS AIDL 接口

8.2.1 IGfs

Diag 模块使用的故障服务核心接口：

```

1 interface IGfs {
2     oneway void reportOccur(in DiagFault event);
3     oneway void reportRecovery(int eventId, long timestamp);
4     oneway void subscribe(int eventId, IGfsSubscriber subscriber);
5     oneway void subscribes(in int[] eventIds, IGfsSubscriber subscriber);
6     oneway void unsubscribe(int eventId, IGfsSubscriber subscriber);
7     oneway void unsubscribes(in int[] eventIds, IGfsSubscriber subscriber);
8     List<DiagFault> queryFaultsAll();
9     DiagFault queryFaultInfo(int eventId);
10    int[] queryFaultCount(int eventId);
11    List<DiagFault> queryFaultsByTime(long beginTime, long endTime);
12 }

```

8.2.2 IDiagOtaManager

Diag OTA 模块使用的升级管理接口：

IDiagOtaManager.aidl

```

1 interface IDiagOtaManager {
2     void registerCallback(IDiagOtaCallback callback);
3     void unregisterCallback();
4     oneway void reportResult(in DiagOtaResponse response);
5     oneway void confirmBlockReceived(String taskId, boolean success);
6     oneway void releaseChannel(String taskId, boolean success);
7 }

```

8.3 Cornerstone AIDL 接口

8.3.1 ICornerStoneService

Maint 模块使用的维护服务接口：

ICornerStoneService.aidl

```

1 interface ICornerStoneService {
2     void submitEvent(in Event event);
3     void registerCallBack(int eventId, ICallback callback);
4     void unregisterCallBack(int eventId, String processName);
5     void mark(in Trigger trigger);
6     void onReceiveMessage(String content);
7     void uploadForEngineMode();
8     void downloadForEngineMode();
9     void captureAllDisplaysForUser(String directory, String fileName);
10    void shutdown(long timeout);

```

九、libgdc 客户端库

 **libgdc** 是独立于 CST SDK 的底层客户端库，提供日志/数据订阅和命令发送能力，适用于需要直接与 GDC 服务交互而不经过程序封装的场景。

9.1 两种传输模式

✓ Socket 模式（默认）

通过 Unix Domain Socket 与 **gdc** 通信。适合本地 CLI 工具和独立进程使用。

✓ Binder 模式

通过 **IGdcLibService** AIDL 接口与 **gdc** 通信。适合需要 Binder 级别安全隔离的应用。

9.2 API 参考

libgdc API

```
1  #include <libgdc.h>
2
3  // ===== 日志订阅 =====
4  // Socket 模式
5  void* handle = libgdc_subscribe_log("cluster", "tombstone", my_log_callback);
6  // Binder 模式
7  void* handle = libgdc_subscribe_log_binder("cluster", "tombstone",
8  my_log_callback);
9  // 取消订阅
10 libgdc_unsubscribe_log(handle);
11
12 // ===== RawData 订阅 =====
13 void* handle = libgdc_subscribe_rawdata("cluster", "coredump",
14 my_rawdata_callback);
15 libgdc_unsubscribe_rawdata(handle);
16
17 // ===== 命令响应订阅 =====
18 void* handle = libgdc_subscribe_cmd_resp(my_cmd_resp_callback);
19 libgdc_unsubscribe_cmd_resp(handle);
20
21 // ===== 发送命令 =====
```

```
20 libgdc_send_command("cluster", "ls /tmp");
```

9.3 回调函数类型

libgdc 回调类型

```
1 // 日志回调
2 typedef void (*dispatch_log_func)(
3     const char* subsys_name,
4     const char* tag,
5     const char* log
6 );
7
8 // RawData 回调
9 typedef void (*dispatch_rawdata_func)(
10     const char* subsys_name,
11     const char* tag,
12     int pkg_id,           // 包序号
13     int pkg_total,       // 总包数
14     const uint8_t* rawdata,
15     int rawdata_size
16 );
17
18 // 命令响应回调
19 typedef void (*dispatch_cmd_resp_func)(
20     const char* subsys_name,
21     const char* cmd,
22     const char* cmd_resp
23 );
```

十、常见问题（FAQ）

Q1: 调用 API 返回 ERR_SERVICE_OFFLINE 怎么办？

SDK 内置了服务自动重连和离线缓存机制。服务恢复后缓存数据会自动 flush，开发者无需额外处理。但需注意：

- Maint Event 队列容量为 500，Mark 队列容量为 1000，超出会丢弃
- Debug 日志按 tag 分组缓存，每个 tag 独立发送线程
- Diag 故障会缓存并在服务恢复后自动重发

Q2: PAD 设备能否使用 Debug 模块?

不能。Debug 模块在 PAD 设备（`gua1_pad`）上编译时即被排除。调用 Debug API 会导致链接错误。PAD 设备请使用 Diag 和 Maint 模块。

Q3: Java SDK 的 CstDiag 为什么调用失败?

Java 层 `CstDiag` 内置了 GUA 签名验证。仅 GUA 签名的 APK 可以正常调用。如果您的 APK 不是 GUA 签名，请联系 GUA 团队获取授权或改用 C/C++ SDK。

Q4: Mark 上报频率过高被丢弃怎么办?

Maint 模块对 Mark 实施令牌桶限流（1 次/秒/EventID，桶容量 100）。建议：

- 合并高频 Mark 为低频批量上报
- 使用 Event 替代 Mark（Event 无限流）
- 超频时 SDK 会自动上报 `eventId= 100010006` 的告警

Q5: 如何选择 Socket 模式和 Binder 模式的 libgdc?

Socket 模式（默认）

- 适合 CLI 工具
- 无需 Binder 环境
- 自动重连
- 函数名无后缀

Binder 模式

- 适合 Android 应用
- Binder 级别安全隔离
- 自动重连 + 死亡通知
- 函数名 `_binder` 后缀

Q6: OTA 升级流程中如何处理异步事件?

OTA 事件处理支持同步和异步两种方式：

- **同步**：在 `onEvent()` 回调中直接处理并返回 `DiagOtaResponse`（`resultCode` 非 `RESULT_PENDING`）
- **异步**：在 `onEvent()` 中返回 `resultCode = RESULT_PENDING`，处理完成后调用 `CstDiagOta::reportResult()` 上报最终结果

十一、配置文件参考

11.1 GDC 配置文件

GDC 服务的子系统通道配置通过 XML 文件定义：

配置文件	用途
<code>gdc_ivi.xml</code>	IVI 侧子系统通道配置（Cluster 用 SHM + Node + PStore，ADSP 用 HAL）
<code>gdc_cluster.xml</code>	Cluster 侧子系统通道配置
<code>shm.xml</code>	Cluster 共享内存布局配置（各区域偏移和大小）

11.2 GFS 配置路径

路径	说明
<code>/data/vendor/gfs/stats/gfs_faults.db</code>	故障 SQLite 数据库
<code>/data/vendor/gfs/evtfiler/</code>	故障事件文件目录

11.3 Cornerstone 配置文件

配置文件	用途
<code>components.json</code>	功能组件开关（logstorage, events, mark, upload 等）
<code>events.json</code>	事件定义（类型、延迟、动作、插件、优先级）
<code>actions.json</code>	动作定义（命令/脚本/服务/函数）
<code>env.json</code>	环境配置（服务器地址、磁盘限制、线程池）
<code>features.json</code>	队列优先级、logstorage 启用列表
<code>logstorage.json</code>	日志源定义（gtag, gtype, readType, writeType, 路径, 大小限制）
<code>network.json</code>	上传间隔和批次配置
<code>export.json</code>	USB 导出配置（签名验证、空间检查、时间范围）

十二、版本历史

版本	日期	变更说明
v1.0	2026-06	初始版本，包含 Debug / Diag / Maint 三大模块完整接口说明

(注：部分内容可能由 AI 生成)